Knowledge-Based Systems 326 (2025) 114064

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

LLM-LADE: Large language model-based log anomaly detection with explanation

Zhiwei Zhang [a], Saifei Li [a,*], Lijie Zhang [b], Jianbin Ye [a], Chunduo Hu [a], Lianshan Yan [a]

[a] College of Information Science and Technology, Southwest Jiaotong University, Chengdu, 611756, China
[b] Institute of Technical Physics of the Southwest, Chengdu, 610041, China

ARTICLE INFO

Keywords:
Log anomaly detection
Log analysis
Large language model
Fine-tuning
Online optimization

ABSTRACT

The increasing complexity of software systems and the exponential growth of log data have made anomaly detection and fault localization from massive logs a critical research area. However, existing log anomaly detection methods struggle to capture context-dependent semantic relationships present in unstructured logs and lack explainable decision-making mechanisms, substantially restricting their applicability in complex systems. The present study proposes LLM-LADE, a novel large language model (LLM)-based framework for log anomaly detection and explanation. LLM-LADE introduces a three-stage coordinated architecture encompassing 'data augmentation - parameter-efficient fine-tuning (PEFT) - online optimization', representing the first implementation of LLMs for simultaneous anomaly discrimination and explanation generation in log sequences. The framework overcomes the interpretability limitations of conventional approaches through a joint learning objective of label-cause pairs. An incrementally matched knowledge base mechanism is designed to enable zero-parameter-update online optimization via real-time capture of misjudged samples, ensuring continuous adaptability to dynamic logging environments. Experimental results on three benchmark datasets demonstrate that LLM-LADE outperforms current state-of-the-art algorithms in detection performance, with explanations generated exhibiting professional-grade usability in terms of both informativeness and readability.

1. Introduction

The increasing of cloud computing and distributed systems has led to an exponential growth in log data volume from modern software systems, accompanied by increasing structural and content complexities [1]. Traditional manual analysis methods relying on regular expression matching suffer significant limitations in terms of real-time processing capability and scalability, particularly in log anomaly detection, wherein manual screening and analysis have become increasingly challenging. Consequently, deep learning-based, automated log anomaly detection has emerged as a research focus, aiming to identify potential anomalous patterns in massive log volumes through data-driven approaches [2,3], thereby intending to enhance detection efficiency and reduce manual analysis burdens.

Current deep learning-based solutions predominantly employ Long Short-Term Memory (LSTM) [4] and Bidirectional Encoder Representations from Transformers (BERT) [5] architectures, typically categorized into two technical paradigms: (1) binary classification-based methods (typically employing supervised learning approaches) [6–13], and (2) sequence generation-based methods (generally utilizing semi-supervised methodologies) [14–20]. The former trains classifiers on extensive log datasets to learn anomaly discrimination rules, whereas the latter models normal log sequence generation patterns to detect anomalies by verifying whether target logs constitute valid subsequent entries or masked tokens. However, these methods generally suffer superficial semantic understanding. LSTM-based models struggle to capture deep semantic relationships in log text, and BERT-based architectures suffer limited capability in modeling domain-specific log knowledge despite their inherent semantic encoding potential. More critically, existing research predominantly focuses on accuracy improvement while neglecting the systematic consideration of detection interpretability, which holds a crucial practical value for security personnel in rapid root cause localization [21]. To address this gap, LogExpert [22] constructs a stack overflow (SO)-based knowledge repository combined with large language models (LLMs) for error log explanation. Although this method provided partial anomaly interpretations, it suffered limited coverage of SO-based knowledge and required pre-identification of error-type logs. Logs2Graphs [23] implemented graph-based anomaly detection with

* Corresponding author.
E-mail addresses: zhangzhiwei@my.swjtu.edu.cn (Z. Zhang), lisaifei@swjtu.edu.cn (S. Li), 569756477@qq.com (L. Zhang), 496984575@qq.com (J. Ye), hcd1123456@163.com (C. Hu), lsyan@swjtu.edu.cn (L. Yan).

https://doi.org/10.1016/j.knosys.2025.114064
Received 18 March 2025; Received in revised form 3 June 2025; Accepted 4 July 2025
Available online 7 July 2025
0950-7051/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

node-level anomaly score decomposition for explanation generation. However, this approach suffered interpretability challenges in practical applications, particularly considering that most log anomalies originate from multi-node interactions rather than from single-node deviations [6].

The past few years have witnessed remarkable advancements in LLMs within the domain of natural language processing, with successful applications spanning finance [24], healthcare [25], and education [26]. Emerging studies [27–29] have demonstrated the potential of LLMs in log analysis tasks, wherein carefully engineered prompts can guide the identification of anomalies in logs. However, the limited cybersecurity background knowledge possessed by LLMs leads to substantially elevated false-positive rates when handling multi-threaded logs and ambiguous contextual scenarios. These limitations underscore the necessity of model fine-tuning to enhance anomaly detection capabilities through log-specific knowledge acquisition.

Although fine-tuning-based approaches [7,30–33] have delivered superior performance over prompt-only methods in log anomaly detection, they suffer two critical challenges in real-world deployment scenarios. First, fine-tuned models often experience a dramatic decrease in detection performance when exposed to evolving log structures [7,31]. Although retraining or adaptation can partially mitigate this issue, these solutions require significant computational resources and are often not feasible in real-world production environments. Second, real-world log datasets are typically highly imbalanced, with anomalies constituting only a small fraction of the data. This class imbalance leads to biased model training and poor recall in anomaly detection [32,33]. While oversampling techniques are commonly used to address this issue, they can cause the model to memorize rather than generalize rare anomalies, and may substantially increase training costs [33]. These limitations underscore the need for a more adaptable and imbalance-aware log anomaly detection framework.

As is well recognized, training models to directly determine whether log sequences are abnormal constitutes a binary classification task. However, directly inputting complete label-cause pairs (i.e., a label for a log sequence and its explanatory cause) into the model may interfere with its comprehension, hindering focus on critical label information. Furthermore, anomaly label distributions are typically highly imbalanced, with anomalous data potentially comprising only 0.5–15 % of the dataset [2]. To mitigate data imbalance challenges, MLog [6] balances class distributions through sampling during training, which does not reflect real-world scenarios. Most studies [8,11,14,15,19,20,33,34] adopt an 80 % training data split from the dataset. Nevertheless, local training on such large-scale data becomes impractical for LLMs owing to their massive parameter sizes.

To address the aforementioned limitations, the present study proposes LLM-LADE, a LLM-based log anomaly detection method. Specifically, raw log data are initially processed using existing log parsing techniques, following which the logs parsed are grouped. A small number of log sequences with labels are manually annotated with corresponding causes. By designing appropriate data augmentation prompts, we leverage the few-shot learning capability of GPT-4 to construct the required training dataset, thereby minimizing manual labeling efforts. The labels of log sequences and their corresponding causes are organized as label-cause pairs to serve as training objectives. Subsequently, the LLaMA3-8B model is supervisedly fine-tuned using the low-rank adaptation (LoRA) algorithm, enabling the model to detect anomalies in new log sequences and generate explanatory causes. Finally, a knowledge base-based feedback workflow is established to record mispredicted log sequences into the knowledge base, facilitating an online error correction mechanism to enhance subsequent model inference. To further address the challenges of data imbalance and computational resource constraints, LLM-LADE optimizes the model training process by adjusting the loss function, guiding the model to learn key features of the target task, thereby effectively alleviating both data imbalance and computational resource consumption. Compared with other methods, LLM-LADE effectively leverages the powerful semantic understanding capabilities of LLMs, requiring only a small amount of training data to deliver superior performance over alternative methods.

The contributions of this paper are summarized as follows. First, a novel LLM-based framework, named LLM-LADE, is introduced for log anomaly detection and explanation. This framework aligns with log analysis objectives through instruction tuning while employing a three-stage collaborative approach comprising data augmentation, parameter-efficient tuning (PEFT), and online optimization. To the best of our knowledge, it represents the first framework capable of simultaneously addressing both the log anomaly detection and explanatory generation tasks through LLM integration. Second, a text matching-based incremental knowledge base is designed to enable the real-time capture of misjudged samples and dynamic updates, achieving online optimization of detection accuracy without requiring adjustments to model parameters.

The remainder of this paper is organized as follows: Section 2 reviews the related work, Section 3 discusses LLM-LADE, and Section 4, the experimental design and offers a comparative analysis of the performance of the model. Section 5 summarizes the present study.

2. Related work

2.1. Log data preprocessing

As unstructured records of system runtime behaviors, logs require effective parsing as a fundamental prerequisite for anomaly detection [35]. Existing studies [6,7,9,11,14–19] have predominantly adopted a two-stage paradigm for 'parsing-detection': unstructured text is first converted into structured event sequences through log parsing that extracts static templates and dynamic variables, followed by anomaly detection. Current log parsing methods are mainly grouped into three technical categories: (1) heuristic rule-based approaches (e.g., Drain [36] and Brain [37]), which construct parsing rules using domain knowledge and implement template matching via data structures such as prefix trees or longest common subsequences; (2) semantic analysis-based methods (e.g., LogPPT [38] and UniParser [39]), which train bidirectional LSTM or pretrained language models to learn semantic information from log messages for template-parameter classification; and (3) pattern-based approaches (e.g., ULP [40]), which identify common elements in logs primarily through frequency or similarity analysis.

Notably, log parsing quality critically influences the performance of log anomaly detection [8]. Through manual parsing error rectification, NeuralLog [20] exhibited 25 % F1-score improvement on the BGL dataset. While log parsing methodology falls beyond the primary scope of this study, given the inherent capability of LLMs to process unstructured log information via semantic comprehension, the LLM-LADE framework innovatively incorporates log parsing as an optional preprocessing module. Through controlled variable experiments, we quantitatively evaluate the marginal contribution of log parsing to detection performance.

2.2. Log anomaly detection models

Early research predominantly employed statistical methods such as principal component analysis (PCA) [41] and invariant mining (IM) [42], which performed anomaly detection by constructing session windows to extract counting features of log templates. Although these approaches could detect certain anomalous logs, they suffered from two critical limitations: (1) semantic information loss owing to one-hot encoding schemes and (2) inadequate modeling of temporal dependencies in log sequences. Aiming at capturing both semantic and temporal patterns in log sequences, deep learning-based methods have been widely adopted for log anomaly detection, primarily categorized into sequence generation approaches and binary classification methods.

Sequence generation-based approaches train deep learning models to learn normal log sequence patterns, flagging deviations from these

patterns as anomalies. DeepLog [14] pioneered LSTM networks for the prediction of subsequent log template IDs, whereas LogAnormaly [15] enhanced this by integrating quantitative sequence features. LogBERT [16] and LAnoBERT [17] adopted BERT architectures for the prediction of masked log template ID. Similarly, LogFit [18] utilized RoBERTa [43] to extract semantic features for masked token prediction. FastLogAD [19] implemented a dual-phase training strategy combining replaced token detection and hypersphere separation learning to compute anomaly scores from log sequence embeddings, enabling threshold-based anomaly judgment. RAPID [34] detected anomalies by retrieving and evaluating the similarity between query and normal logs without requiring training on task-specific log datasets.

Binary classification methods directly provide probability estimates for normal/anomalous log sequences through supervised training. LogRobust [8] combined pretrained FastText [44] embeddings with TF-IDF weighted aggregation, employing Bi-LSTM networks with attention mechanisms for detection. Analogously, LayerLog [12] integrated word, log, and logSeq-level semantic representations for Bi-LSTM training. PLELog [9] estimated probabilistic labels for unannotated log sequences by using attention-enhanced GRU networks. Logsy [10] modified objective functions via hyperspheric decision boundaries to derive anomaly scores. LogFormer [11] proposed a log-attention module to compensate for the semantic information loss during log parsing and pre-training a transformer-based encoder for anomaly detection. NeuraLog [20] leveraged BERT-derived semantic representations of normalized log sequences for transformer-based classification. BTCNLog [13] employed dictionary-based parsing to retain critical parameters, utilized BERT-based semantic encoding and clustering for label estimation, and trains a bidirectional temporal convolution network model to generate the output.

Recent advances also include graph-based approaches. Logs2Graphs [23] transforms log data into directed and weighted graphs, utilizing a one-class graph convolutional network(GCN) for unsupervised anomaly detection. GLAD [45] employs few-shot learning to automatically extract entities from log messages and constructs dynamic graphs, combining GCN and Transformer to detect overall anomalies. In addition, graph contrastive learning methods, such as CI-GCL [46] and TACL [47], have further improved the robustness of graph representations by introducing augmentation strategies that preserve community structures or integrate temporal and structural views.

The rapid advancement of LLMs built on transformer decoder architectures has spurred increasing exploration of their applications in log anomaly detection. LogGPT [27] and LogPrompt [28] employ task-specific prompts to query ChatGPT for log sequence anomaly identification; however, their effectiveness is constrained by the context window limitations of ChatGPT, when processing lengthy log sequences. Egersdoerfer et al. [29] developed a summarization-driven memory update mechanism that integrated historical log context with current log windows for anomaly diagnosis. RAGLog [48] employed a retrieval-augmented generation (RAG) framework to analyze logs by comparing target log sequences against pre-stored normal log sequences. Prompt engineering-based methods heavily rely on the intrinsic knowledge possessed by LLMs, resulting in suboptimal detection performance on log datasets. Owl [30] enhanced detection efficacy through supervised fine-tuning on cybersecurity datasets, and Fatemeh Hadadi et al. [7] achieved better performance than prompt engineering via ChatGPT-3 fine-tuning with log data. LogLLM [33] extracts semantic representations from log sequences using a pre-trained BERT model and employs a projection module to align the representation space with that of LLaMA. The aligned vectors are then used to fine-tune LLaMA for anomaly classification. However, these approaches solely focus on anomaly detection without leveraging the reasoning capabilities of LLMs for explanatory generation.

By contrast, LLM-LADE introduces a unified framework that leverages LLMs to jointly model label-cause pairs through a shared learning objective. The design is able to simultaneously output anomaly detection results and interpretable causal explanations, addressing the issues of detection accuracy and diagnostic practicality via a single framework. Compared with a previous work that is limited to single-task optimization, LLM-LADE combines anomaly identification and cause analysis, thereby improving the practicality of the automated troubleshooting process.

3. Method

This section elaborates on the proposed log anomaly detection and explanation framework, LLM-LADE. As shown in Fig. 1, this methodology establishes a triphasic collaborative system ('data augmentation - PEFT - online optimization') that harnesses the semantic comprehension and few-shot learning capabilities of LLMs, effectively addressing the limitations of traditional methods in semantic modeling and interpretable decision-making. Specifically, this method implements log data augmentation through task-driven prompt engineering, achieves domain knowledge transfer via PEFT techniques, and incorporates a dynamic knowledge base to enable continuous model optimization. The following subsections detail each component.

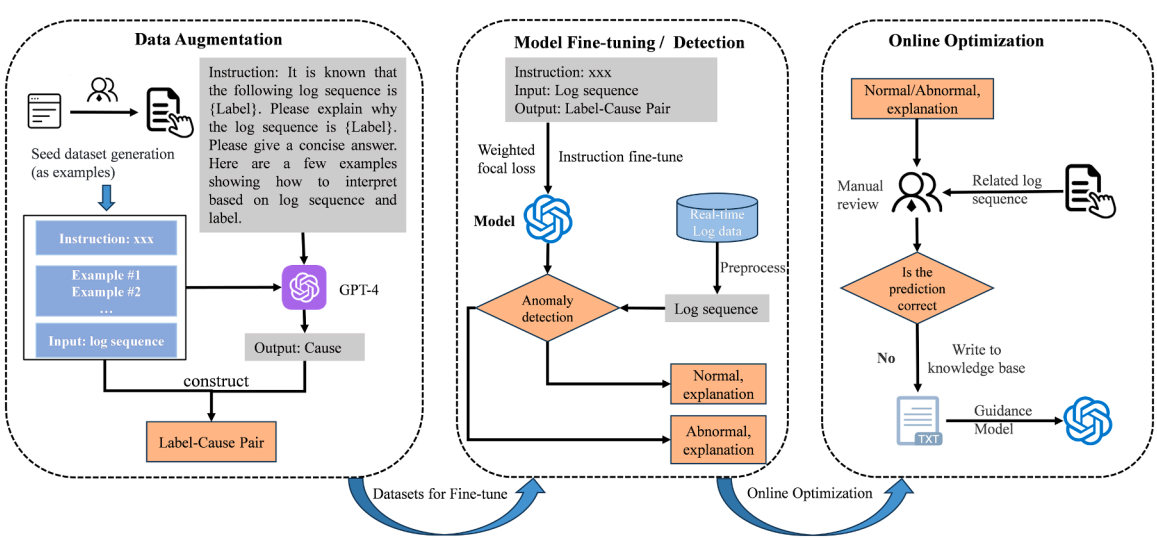

Fig. 1. Overview of LLM-LADE framework.

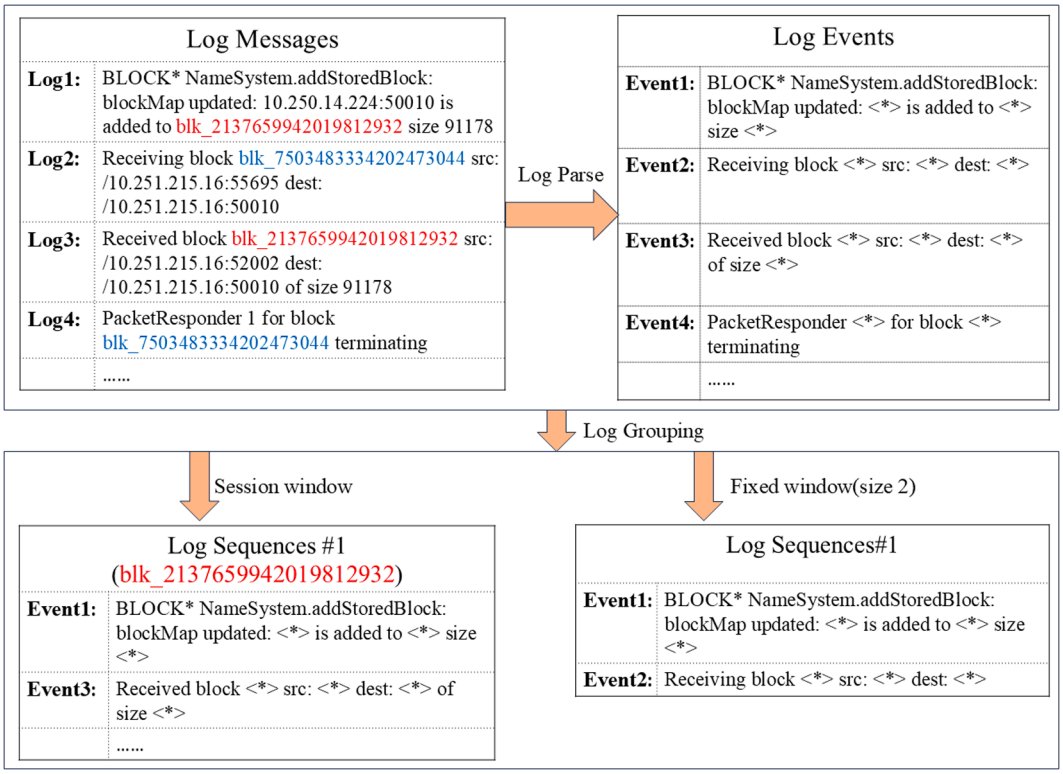

Fig. 2. Log parsing and log grouping process.

3.1. Log data augmentation

As semi-structured textual data, logs require parsing to transform them into structured representations-a critical preliminary phase for most log anomaly detection tasks [35]. Herein, a log parsing module is implemented as an optional component, after which the log sequences undergo grouping-based preprocessing irrespective of their parsing activation status, with the workflow shown in Fig. 2. For the HDFS dataset, session segmentation is performed based on the block_id field to maintain spatiotemporal continuity of operational workflows, whereas fixed-window mechanisms are applied for temporal chunking in BGL and Thunderbird system logs.

Existing log datasets predominantly utilize manual annotation for labeling anomaly states but lack cause descriptions. Prior approaches trained models exclusively on label-only data to develop anomaly classification capabilities. Although such models could predict anomaly occurrence, the absence of causal labels prevented the generation of actionable diagnostic evidence, severely limiting practical utility in fault remediation and root cause localization.

Aiming at addressing this limitation, we propose an LLM-powered data augmentation strategy to construct an enriched dataset with causal explanations. This implementation involves three phases: first, 30 log sequences are randomly sampled from the original dataset and annotated with anomaly causes by domain experts to form a seed dataset. Second, a structured prompt template is engineered to guide GPT-4 for data augmentation, consisting of the following three components: (1) task instructions explicitly mandating the model to infer anomaly existence and causal mechanisms from log sequences; (2) dynamic exemplars incorporating five randomly selected human-annotated input-output pairs per query to stimulate few-shot reasoning; and (3) target input containing complete log sequences awaiting analysis. To mitigate LLM hallucinations, model outputs are subjected to manual verification and selective correction. Finally, label-cause pairs are constructed by associating log sequence labels with GPT-4-generated explanations, serving as learning objectives during fine-tuning. Table 1 presents representative examples of the constructed label-cause pairs. Each pair consists of a binary anomaly label and a corresponding explanation describing the cause.

Table 1
Examples of constructed label-cause pairs.

Log sequence (Excerpt)	Label-cause pairs
Receiving block * src: * dest: *, BLOCK* NameSystem.allocateBlock: *, writeBlock * received exception java.io.IOException: Could not read from stream	Abnormal-The log sequence is abnormal because it indicates that there was an issue reading from a stream, causing an IOException during the allocation and writing of a block in the NameSystem.

3.2. Model fine-tuning

For model development targeting log anomaly detection and explanation tasks, this study employs a LoRA-based, supervised fine-tuning framework with the LLaMA3-8B model,[1] integrated with a dynamically weighted loss function design to effectively address class imbalance issues. Although full fine-tuning generally achieves better performance in high-resource scenarios [49], we adopt LoRA to strike a practical balance between model accuracy and hardware efficiency. This makes our approach more suitable for real-world deployment environments wherein computational resources are limited.

LoRA introduces the low-rank decomposition matrices $\mathbf{B} \in \mathbb{R}^{d\times r}$ and $\mathbf{A} \in \mathbb{R}^{r\times d}$ into the transformer layers of the pre-trained model. By approximating the full-parameter fine-tuning update matrix $\Delta\mathbf{W}$ as $\Delta\mathbf{W} = \mathbf{BA}$, this parameterization reduces trainable parameters from $\mathcal{O}(d^2)$ to $\mathcal{O}(2dr)$ where $r \ll d$, enabling efficient fine-tuning while keeping the original weights $\mathbf{W}_0$ frozen. The parameter update process is formalized

[1] https://huggingface.co/meta-llama/Meta-Llama-3-8B

as follows:

$$\mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{BA} \tag{1}$$

where $\mathbf{W}_0 \in \mathbb{R}^{d\times d}$ denotes the original parameters, and $\mathbf{B} \in \mathbb{R}^{d\times r}$ with $\mathbf{A} \in \mathbb{R}^{r\times d}$ represent the low-rank adaptation weight matrices.

In our framework, supervised fine-tuning is formulated as a conditional sequence generation task, wherein the model learns to generate a binary anomaly label and a corresponding natural language explanation based on the input log sequence. Although this task differs from standard next-token prediction in classical language modeling, it still follows a token-by-token generation process, which can be effectively optimized using a cross-entropy loss by minimizing the discrepancy between predicted token distributions and ground-truth outputs. The original cross-entropy loss function is shown in Eq. (2):

$$\mathcal{L} = -\sum_{c=1}^{M} y_c \log(\hat{y}_c) \tag{2}$$

where M denotes the total number of classes, $y_c \in \{0, 1\}$ represents a one-hot encoded vector indicating the true class (with $y_c = 1$ if the target class is c, and 0 otherwise), and $\hat{y}_c \in (0, 1)$ is the predicted probability for class c.

However, the model is trained to generate a binary anomaly label followed by a multi-sentence explanation for each log sequence. Applying the standard cross-entropy loss directly would assign an equal weight to both components, allowing the longer explanatory text to dominate the loss signal and hinder the effective learning of the label. To prioritize label learning, we designed a weighted cross-entropy loss function that introduces the coefficient β to control the contribution of multi-sentence generation losses. This prevents long-sequence dominance while assigning higher weights to label information and lower weights to label-cause pairs. The weighted loss function is shown in Eq. (3):

$$\begin{aligned}\mathcal{L}_{\text{weighted}} = \quad & \alpha \cdot \underbrace{\left(-\sum_{i=1}^{V} y_{t,i} \log \hat{y}_{t,i}\right)}_{\text{Label loss}} \\ & + \beta \cdot \underbrace{\left(-\frac{1}{N_r} \sum_{j=1}^{N_r} \sum_{i=1}^{V} y_{r,j,i} \log \hat{y}_{r,j,i}\right)}_{\text{Cause loss}}\end{aligned} \tag{3}$$

where $\alpha, \beta \in [0, 1]$ with $\alpha + \beta = 1$ denote the weighting coefficients for labels and label-cause pairs respectively. Here V denotes the vocabulary size, N_r represents the word count in label-cause pair sentences, $y_{t,i}$ and $\hat{y}_{t,i}$ correspond to the ground-truth and predicted distributions for the ith token in label components, while $y_{r,j,i}$ and $\hat{y}_{r,j,i}$ represent the distributions for the ith word in the jth position of cause components.

To further address extreme class imbalance in log data wherein normal logs considerably outnumber anomalies, direct application of the cross-entropy loss causes model bias toward majority class prediction while neglecting minority classes. To mitigate this issue, we integrated a focal loss [50] into our loss design. This was not introduced as a novel contribution, rather as a practical component of our multi-objective learning framework. Specifically, we applied focal loss to the label prediction subtask to enhance the sensitivity of the model to rare anomalies while maintaining the standard cross-entropy loss for the explanation generation subtask. The modulating factor γ in the focal loss dynamically reduces the contribution of easily classified (normal) samples, allowing the model to focus more on difficult and underrepresented instances. The final loss function is formulated in Eq. (4):

$$\begin{aligned}\mathcal{L}_{\text{final}} = \quad & \alpha \cdot \underbrace{\sum_{i=1}^{V} \alpha_t (1 - \hat{y}_{t,i})^{\gamma} y_{t,i} \log \hat{y}_{t,i}}_{\text{Focal Loss (Labels)}} \\ & + \beta \cdot \underbrace{\left(-\frac{1}{N_r} \sum_{j=1}^{N_r} \sum_{i=1}^{V} y_{r,j,i} \log \hat{y}_{r,j,i}\right)}_{\text{CE Loss (Causes)}}\end{aligned} \tag{4}$$

where $\alpha_t \in [0, 1]$ is the class weighting parameter for labels, $\gamma \geq 0$ denotes the focal modulation factor, and $\hat{y}_{t,i} \in (0, 1)$ represents the predicted probability for the ith token in label components. The weighting coefficients satisfy $\alpha + \beta = 1$ with $\beta = 1 - \alpha$, V is the vocabulary size, and N_r indicates the word count in cause sentences.

This progressive loss design, which transitions from standard cross-entropy to weighted cross-entropy and finally incorporates a focal loss, allows our framework to effectively balance learning between short classification labels and long explanatory sequences while enhancing robustness in scenarios of class imbalance.

3.3. Log anomaly detection

During the detection phase, the fine-tuned LLM-LADE model performs online log anomaly detection via the following workflow: first, raw logs are subjected to structured preprocessing via predefined log parsing rules and event grouping strategies, converting them into standardized event sequences. These processed sequences are then fed into the LLM-LADE model, which integrates its pre-trained, general semantic encoding capabilities, and domain-specific discriminative knowledge acquired during fine-tuning to execute anomaly pattern recognition. Simultaneously, the model outputs binary anomaly classification labels and corresponding interpretability analyses.

3.4. Online optimization

To address the detection performance deterioration caused by concept drift induced by the continuous evolution of log patterns [51], the present study designs an online optimization framework incorporating human expert feedback. This mechanism enables model performance adaptation through dynamic knowledge base updates, as shown in Fig. 3. Specifically, after the detection model outputs a label-cause pair tuple, domain experts evaluate result credibility based on log contextual semantics and system operational states. Misclassified cases (false positives/negatives) are stored as key-value pairs comprising raw log sequences and corrected labels in the incremental knowledge base. A chunked storage mechanism ensures that each data unit strictly corresponds to individual log sequences with human-annotated labels.

Traditional vector similarity retrieval may induce semantic ambiguity in scenarios with high semantic similarity across log sequences. A token-based matching strategy is employed to mitigate this limitation. Label prompting is triggered only when the tokenized input log sequence exactly matches an entry in the knowledge base, thereby effectively preventing interference from semantically similar but unannotated log sequences. Matched log sequences prioritize corresponding knowledge base annotations for model decisions, whereas unmatched cases rely on pretrained knowledge for autonomous inference. This hierarchical decision-making mechanism maintains high utilization of expert annotations while preserving the generalization capabilities of the model for unseen patterns.

4. Experiments

Comprehensive evaluations were conducted on real-world datasets to validate the effectiveness of the proposed method. Detailed descriptions encompass datasets, experimental configurations, evaluation metrics, and baseline methodologies. Extensive experiments were performed to assess both the critical components and overall performance of LLM-LADE. Furthermore, the marginal contribution of log parsing to detection performance was systematically investigated. The annotated seed datasets and LoRA adapter weights used in our experiments are available at: https://github.com/sleep-zzw-bot/LLM-LADE.

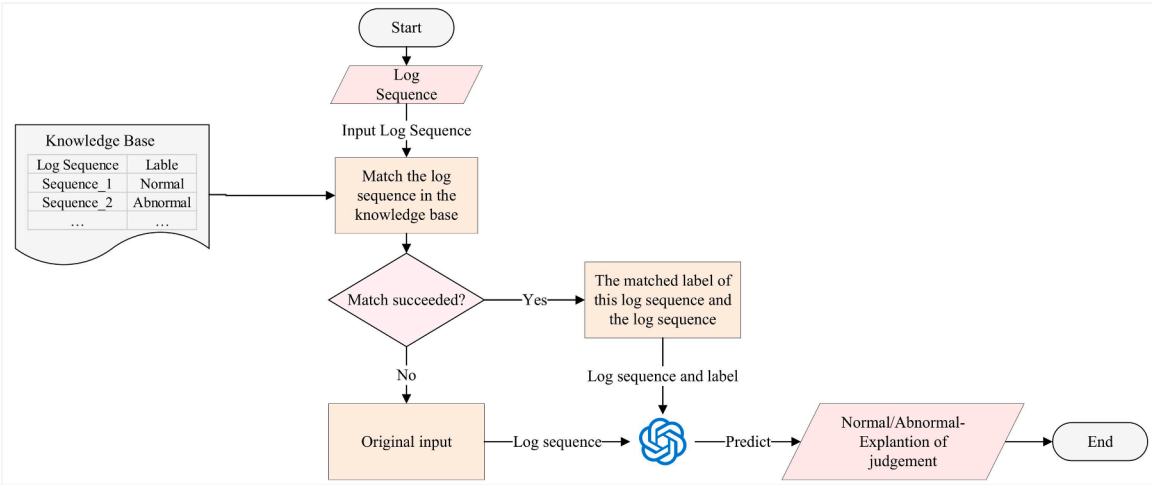

Fig. 3. An online optimization framework incorporating human expert feedbacks.

4.1. Experimental settings

4.1.1. Datasets

Loghub [52] provides multiple real-world log datasets, of which three widely used ones were selected for the present study: HDFS, BGL, and Thunderbird. The remaining datasets in Loghub have rarely been employed as benchmarks for log anomaly detection tasks owing to their lack of manual labeling or insufficient data volume. The details of each dataset are provided in the following paragraphs:

HDFS [53] dataset is a labeled dataset from Loghub; it contains logs generated by running benchmark workloads on a 203-node Hadoop Distributed File System cluster. Each log sequence (segmented by block IDs) is assigned a ground-truth label (normal or abnormal) using manually crafted rules. The tokenized length of the longest sequence across all sessions is 2777 tokens after processing with the LLaMA3-8B tokenizer. This value lies within the maximum input limit (8,192 tokens) of the model, ensuring that all log sequences can be processed without truncation.

BGL [54] dataset is a critical system log dataset from Loghub, comprises logs collected from the Blue Gene/L supercomputer system at Lawrence Livermore National Laboratory. These logs include alert and non-alert messages, where the first column uses "-" to denote non-alert entries and other labels for alerts. Unlike block-based segmentation in HDFS, a fixed window mechanism with a window size of 20 and step size of 20 is employed.

Thunderbird [54] dataset originates from the Thunderbird supercomputer system at Sandia National Labs. Similar to BGL, the first column of Thunderbird logs marks abnormal entries. Owing to the massive volume of Thunderbird logs, only the first 10 million entries were used for experimental efficiency. The same fixed window mechanism as that of BGL is adopted, and a log sequence is labeled abnormal if it contains at least one abnormal log entry.

Unlike prior research works [8,9,14,15,19,20,33,34] that partitioned datasets into 8:2 training set-test set splits, our experiment sampled no more than 20 % of log sequences for training, with the remainder used for evaluation. This choice was mode owing to two practical considerations: (1) the high computational cost associated with the fine-tuning of LLMs (e.g., LLaMA3-8B) and (2) the goal of evaluating the robustness and data efficiency of LLM-LADE under limited supervision, demonstrating the strong generalization ability and practical applicability of our model in low-resource scenarios. Table 2 presents the statistical details of the experimental datasets used.

4.1.2. Experimental setup and metrics

Our experiments were conducted on servers equipped with NVIDIA GeForce RTX 4090 GPUs using CUDA 12.2. During model fine-tuning, we employed a maximum learning rate of 5e-5, optimized with the AdamW [55] optimizer and a cosine annealing learning rate decay strategy. For fine-tuning, the batch size was set to one with gradient accumulation steps of eight, simulating an effective batch size of eight through gradient accumulation. The model was trained for one epoch under this configuration.

The experiment employed precision, recall, and F1-score as performance metrics to evaluate the effectiveness of the model. These metrics are defined as follows:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{5}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{6}$$

$$F_1\text{-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{7}$$

4.1.3. Benchmark methods

To validate the effectiveness of the proposed method, we compared it with 9 widely cited log anomaly detection frameworks featuring publicly available code, including DeepLog [14], LogAnomaly [15], LogBERT [16], LogRobust [8], PLELog [9], NeuralLog [20], LogPrompt [28], RAPID [34], and logLLM [33]. Specifically, based on

Table 2
Dataset statistics for experimental evaluation.

Dataset	Log statistics		Training data			Testing data	
	Log messages	Log seqs	Seqs	% Anomaly	% Train	Seqs	% Anomaly
HDFS	11,175,629	575,061	40,000	2.82	7.0	535,061	2.94
BGL	4,631,261	231,564	40,000	8.18	17.3	191,564	8.07
Thunderbird	10,000,000	500,000	40,000	24.60	8.0	460,000	24.80

Seqs: Log Sequences; % Anomaly: Anomaly Ratio; % Train: Training Ratio.

Table 3
Experimental results on HDFS, BGL and thunderbird datasets.

Methods	HDFS			BGL			Thunderbird		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
Deeplog	0.786	0.750	0.768	0.897	0.828	0.861	0.873	0.996	0.931
LogAnomaly	0.968	0.618	0.755	0.731	0.761	0.741	0.867	0.996	0.927
LogRobust	0.983	0.948	0.946	0.897	0.792	0.841	0.458	0.917	0.611
LogBERT	0.757	0.666	0.709	0.894	0.923	0.908	0.967	0.965	0.966
NeuraLog	0.982	0.988	0.983	0.929	0.913	0.921	0.814	0.933	0.869
PLElog	0.950	0.963	0.957	0.862	0.844	0.853	0.804	0.726	0.762
LogPrompt (ChatGPT)	0.190	0.803	0.308	0.249	0.834	0.384	0.256	**1.000**	0.407
RAPID	**1.000**	0.810	0.890	0.287	**1.000**	0.446	0.833	0.869	0.851
LogLLM	0.987	0.938	0.963	0.929	0.972	0.950	0.998	0.665	0.800
LLM-LADE (ours)	0.983	**0.998**	**0.990**	**0.992**	0.987	**0.989**	**0.999**	0.994	**0.997**

† Best results are highlighted in bold.

the training paradigm, we classify these methods into three categories. (1) Semi-supervised methods (trained only on normal logs): DeepLog, LogAnomaly, LogBERT,[2] and PLELog.[3] We reproduced these methods using their official implementation codes. (2) Supervised methods (trained with labeled normal and abnormal logs): LogRobust, NeuralLog,[4] and LogLLM.[5] These methods were also reproduced using their official source codes under the experimental setups specified in their respective papers. (3) Prompt-based or training-free methods: LogPrompt and RAPID[6] do not require model training. We directly reported the performance of LogPrompt on the BGL dataset from the original paper and reproduced results on HDFS and Thunderbird by using the prompt templates provided. RAPID detects anomalies by assessing the similarity between log sequences and normal samples. We reproduced the results of RAPID by using the authors' official implementation and their original data split settings.

4.2. Evaluation

4.2.1. Anomaly detection performance evaluation

Table 3 presents the experimental results of various methods on the HDFS, BGL, and Thunderbird datasets, with the best results highlighted in bold. These results demonstrated that the proposed LLM-LADE method offered significant advantages in log anomaly detection tasks, achieving F1-scores of 0.990, 0.989, and 0.997 on the HDFS, BGL, and Thunderbird datasets, respectively, outperforming all existing state-of-the-art models. Specifically, on the Thunderbird dataset, containing complex heterogeneous logs, LLM-LADE improved the F1-score by 3.1 % relative to LogBERT, a semi-supervised model. On the HDFS dataset, LLM-LADE matched the performance of the best model at present, NeuralLog. These results indicated that the supervised fine-tuning strategy effectively enhanced the deep semantic understanding of multi-dimensional log patterns in LLM.

On the BGL dataset, LLM-LADE achieved a 3.9 % higher F1-score than that of LogLLM, another recent LLM-based anomaly detection method. This performance advantage stemmed from several key design differences. Specifically, LogLLM formulates anomaly detection as a single-task binary classification problem and addresses class imbalance via data resampling, which may lead to overfitting or distribution shift. By contrast, LLM-LADE adopts a multi-task generation strategy that jointly models anomaly labels and natural language explanations, and it uses a dynamic loss function combining weighted cross-entropy and a focal loss for achieving finer optimization. Moreover, LLM-LADE utilizes a streamlined, end-to-end architecture solely based on the LLaMA model. LogLLM, however, relies on a three-stage pipeline comprising a BERT encoder, a projection module, and an LLaMA model, which may introduce representation mismatches and semantic information loss during projection. This suggests that even within the same category of LLM-based methods utilizing supervised fine-tuning, LLM-LADE delivered superior performance owing to its more effective task formulation and model architecture.

The zero-shot detection method LogPrompt, based on LLMs, achieved high recall rates of 0.834 and 1.0 on the BGL and Thunderbird datasets, respectively, but its precision fell below 30 %, revealing the occurrence of severe false positives when directly leveraging ChatGPT for anomaly detection. This was attributed to the pretrained model's bias in recognizing domain-specific log patterns. Meanwhile, the retrieval-based method RAPID suffered a sharp decline in F1-score (0.446) on the BGL dataset. Analysis demonstrated that most misclassified samples arise from semantically similar warning logs being incorrectly labeled as abnormal, highlighting the inherent sensitivity of retrieval-based methods to data distribution.LLM-LADE employs augmented log data for supervised fine-tuning, effectively leveraging the semantic comprehension capabilities of the LLMs combined with supervised learning mechanisms to precisely recognize log information. To ensure fair comparison, all baseline methods were made to use a consistent log parser (Drain) throughout experiments. Moreover, the knowledge base was not activated during the anomaly detection evaluation.

4.2.2. Validity of explanations generated via LLM-LADE

Our goal was not to reconstruct a ground-truth root cause for each log sequence but to determine whether the model-generated explanation will provide a plausible justification for the predicted label. In the absence of reference root causes, we evaluated whether the explanation logically aligned with both the predicted label and the semantic content of the input log sequence. Given that traditional automated metrics such as Bilingual Evaluation Understudy (BLEU) or Recall-Oriented Understudy for Gisting Evaluation (ROUGE) require fixed references, which are not available in log datasets, we adopted a hybrid evaluation strategy that combines GPT-4 based scoring and expert human review. This approach was supported by the recent studies demonstrating that LLMs can reliably assess the quality of generated content without ground-truth references and often produce judgments highly consistent with human ratings [56–58]. Specifically, the GPT-4 model was employed alongside a review panel of three industry engineers, who were tasked with manually scoring the explanations given by the model. Following Ahmed et al. [59], this evaluation focused solely on usefulness and readability scores, defined as follows:

Usefulness: These scores reflected the depth, specificity, relevance, logical coherence, and practical utility of decision explanations in log analysis. A useful explanation was defined as the one which was detailed, task-specific, logically structured, and practically applicable to log analysis workflows.

[2] https://github.com/HelenGuohx/logbert
[3] https://github.com/YangLin-George/PLELog
[4] https://github.com/vanhoanglepsa/NeuralLog
[5] https://github.com/guanwei49/LogLLM
[6] https://github.com/DSBA-Lab/RAPID

Table 4
Evaluation criteria for explanatory effectiveness of LLM-LADE.

Scores	Usefulness	Readability
1	The explanation is nearly irrelevant or completely incorrect, offering no help in solving the problem.	The text contains many grammatical errors, the content is difficult to understand or completely meaningless, and lacks professionalism.
2	The explanation provided is limited and only marginally relevant in some areas, but not very helpful overall.	The text contains some grammatical errors, some content is difficult to understand, and the professionalism is weak.
3	The explanation provided is detailed and specific to a certain extent, some content is relevant to the problem, the logic is reasonable, and it helps solve the problem.	The text's grammar is correct, the content is generally meaningful, the expression is acceptable, and the professionalism is average.
4	The explanation provided is detailed, specific, highly relevant, logical, and provides great help in solving the problem.	The text is grammatically correct, the content is clear and meaningful, the expression is fluent and relatively professional.
5	The explanation provided is extremely detailed and specific, completely relevant, logically sound, and greatly aids in problem-solving.	The text is grammatically perfect, the content is clear and in-depth, and the expression is very fluent and highly professional.

Table 5
LLM-LADE explanation quality scores under dual GPT-4 and human evaluation.

Method	Normal				Abnormal			
	Usefulness		Readability		Usefulness		Readability	
	Mean	HIP	Mean	HIP	Mean	HIP	Mean	HIP
GPT-4	4.332	0.865	4.379	0.872	4.244	0.813	4.413	0.849
Human Eval.	4.282	0.832	4.452	0.898	4.203	0.804	4.384	0.902

Readability: These scores evaluated text comprehensibility, emphasizing grammatical correctness, meaningfulness of content, professional expression, and ease of understanding.

For evaluation, 200 normal and 200 abnormal logs with correct predictions were randomly selected from the HDFS, BGL, and Thunderbird test sets. The review panel independently scored the usefulness and readability of each sample on a 1–5 scale. To standardize the evaluation process, detailed scoring criteria for each level are presented in Table 4. Two metrics were reported: (1) average score across all samples and (2) high interpretability percentage (HIP), representing the proportion of samples scoring higher than 4.0.

The experimental results, as shown in Table 5, demonstrated that LLM-LADE delivered superior performance in generating explanations for both the normal and abnormal logs under the dual evaluation framework via combining a GPT-4 model and manual review. Across the usefulness and readability dimensions, the average scores for both log types exceeded 4.0, whereas the HIP was consistently maintained above 0.8. These results indicated that the majority of explanations generated were deemed highly helpful and readable both by the GPT-4 model and the review panel.

4.3. Ablation studies

4.3.1. Performance gain of log parsing for LLM-based anomaly detection

To systematically evaluate the contribution of log parsing to LLM-based anomaly detection, the present study compared four typical parsing methods (Drain [36], Brain [37], ULP [40], and UniParser [39]) against raw log processing (Raw) on three benchmark datasets. Experimental results, as shown in Fig. 4, quantified detection performance using F1-scores.

On the HDFS dataset, the Drain parser achieved a substantially higher F1-score of 0.990 compared to those of other parser methods, attributed to its dynamic depth tree structure enabling the precise matching of complex log templates, thereby enhancing anomaly detection accuracy. Notably, raw logs (F1 = 0.971) outperformed Brain, ULP, and UniParser, suggesting that the template abstraction errors during parsing may introduce information loss, making raw logs with fine-grained details more effective in certain scenarios. For the BGL and Thunderbird datasets, performance differences among methods were marginal, indicating limited benefits of log parsing for anomaly detection. This likely stemmed from the inherent capability of the LLM to directly extract meaningful patterns from raw logs without relying on parsing.

The present study quantified the performance gains of log parsing in LLM-based anomaly detection. On HDFS, precise parsing substantially improved detection performance; however, on BGL and Thunderbird, the computational overhead of parsing may have outweighed the marginal performance gains. These findings provided a practical guidance for system design: prioritize evaluating the necessity of log parsing for target log data prior to deploying LLM-based detectors, rather than defaulting to parsing workflows.

4.3.2. Impact of loss functions

To address the prevalent class imbalance in log data, a controlled experiment was designed to compare the anomaly detection performances of the focal loss and standard cross-entropy loss on three benchmark log datasets while maintaining identical network architectures and training strategies. Experimental results as shown in Fig. 5 demonstrate that the focal loss achieved significant performance gains in anomaly detection tasks. Specifically, the magnitude of its advantage exhibited a nonlinear negative correlation with the proportion of anomalies in the dataset.

On the HDFS dataset with an anomaly rate below 3 %, the focal loss achieved a substantially enhanced F1-score. However, on the BGL dataset with an anomaly rate of approximately 8 %, the performance improvement narrowed. On the Thunderbird dataset with an anomaly rate of around 25 %, the focal loss provided negligible benefits. The focal loss intelligently allocated gradient contributions via a modulation factor, suppressing loss weights for easily classified normal samples to mitigate overfitting while amplifying weights for challenging anomalies to enforce deep feature extraction. On Thunderbird, the near-normal

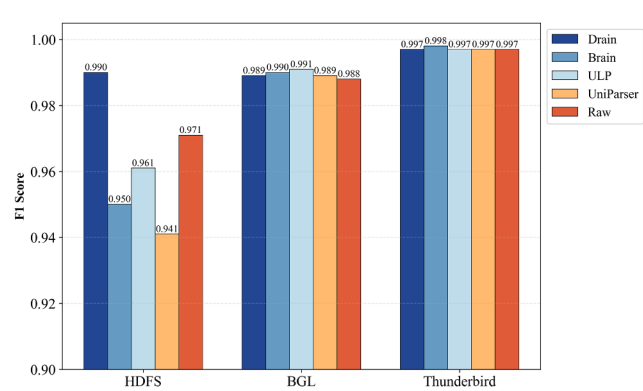

Fig. 4. Impact of Log Parsing on LLM-based Anomaly Detection Performance.

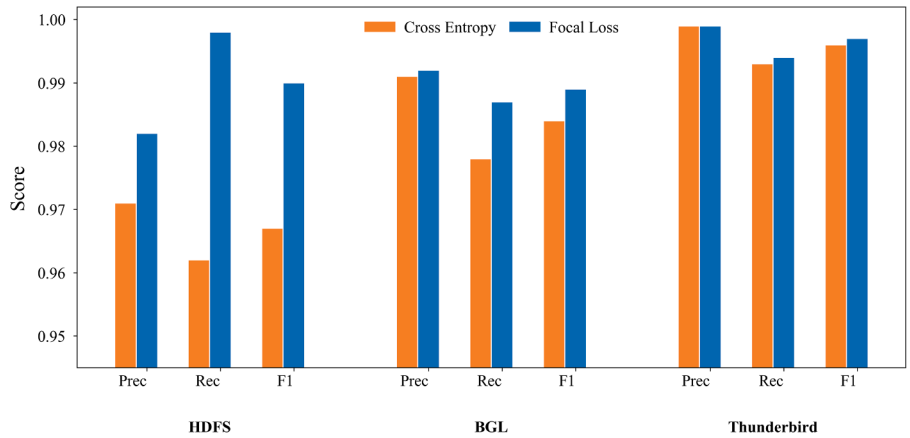

Fig. 5. Comparison of Focal Loss and Cross Entropy on HDFS, BGL and Thunderbird.

distribution of samples reduced the weight modulation by 1.1–1.3 times, diminishing the efficacy of the mechanism.

By proposing a loss function-optimization method based on a dynamic weighting strategy, the present study addressed the issue of label information being overshadowed by cause analysis loss in log anomaly detection tasks. The introduction of the adjustable weighting coefficient α effectively balanced the attention allocation of the model between cause reasoning and label prediction. Comprehensive experiments conducted on three log datasets systematically evaluated the impact of different α values on model performance. The experimental results, as shown in Fig. 6, demonstrate that at $\alpha = 0$ (without the weighting strategy), the F1-scores on HDFS, BGL, and Thunderbird were 0.948, 0.949, and 0.974 respectively, indicating significant deficiencies in learning of label information. With increments in α, all datasets showed notable performance improvements. While HDFS and BGL showed gradual enhancements with higher weighting coefficients, Thunderbird entered a performance saturation state at $\alpha \geq 0.3$, with the stabilized F1-scores primarily attributed to its superior class separability. This performance enhancement revealed that the weighting coefficient induced gradient modulation effects during backpropagation, effectively intensifying the focus of the model on label prediction tasks.

4.3.3. Impact assessment of knowledge base feedback on detection performance

Assuming that manually annotated log labels exhibit gold-standard reliability, log sequences with discrepancies between the predictions made by the model and ground-truth labels are automatically written to an incremental knowledge base, alongside their correct labels. This knowledge base guides the detection process via a non-parametric updating mechanism grounded in precise pattern matching, wherein performance optimization exclusively relies on content retrieval guidance without requiring adjustments to model parameters.

To validate this mechanism, a stratified sampling experiment was conducted on the HDFS and BGL datasets. A total of 100,000 log sequences were randomly selected from each test set. Initial model predictions revealed 64 false positives (FPs) and 5 false negatives (FNs) on HDFS, and 59 FPs and 106 FNs on BGL. After injecting these error samples into the knowledge base, a secondary detection on same test sets achieved 100 % precision and recall on both datasets. This confirmed two core properties of the mechanism: (1) exact pattern-matching retrieval ensures 100 % precision and (2) predictions for non-matching samples remain unaffected.

A total of 50,000 log sequences from the remaining test sets were evaluated to further assess generalization ability of the model. The experimental group utilized the knowledge base derived from the initial 100,000 logs, whereas the control group solely relied on the baseline model. As shown in Table 6, on HDFS, FPs decreased from 17 to 13, FNs reduced from 6 to 5, and the F1-score improved from 0.992 to 0.994. This represents a reduction of 5 prediction errors out of 23 total, corresponding to a 21.7 % decrease in error cases. Similar positive improvement trends were noted on BGL. These results demonstrated that the mechanism preserved the inherent reasoning capabilities of the model

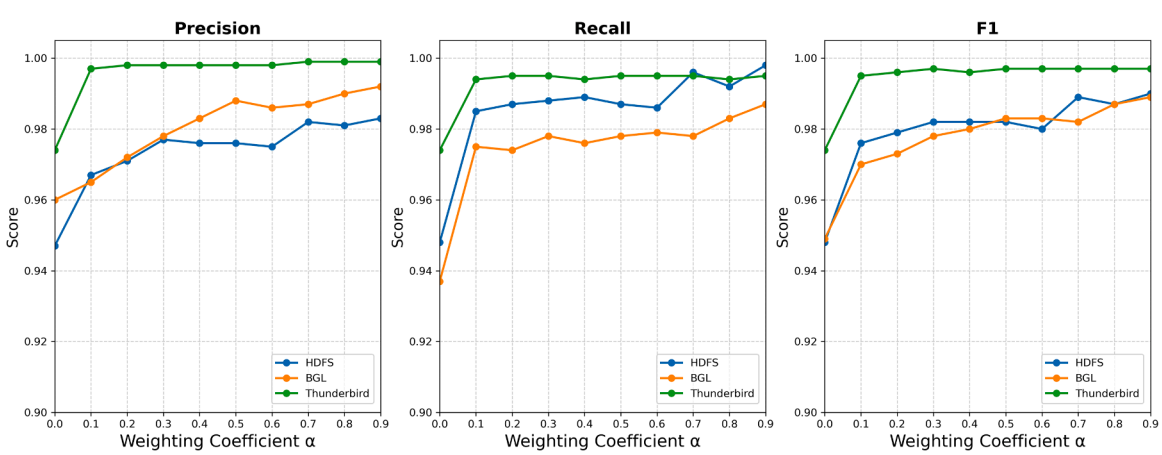

Fig. 6. Effect of Weighted Loss Coefficient on Detection Performance on HDFS, BGL and Thunderbird.

Table 6
Impact of knowledge base on model performance.

Method	HDFS			BGL		
	FP	FN	F1 Score	FP	FN	F1 Score
No knowledge base	17	6	0.992	29	65	0.989
Use knowledge base	13	5	0.994	28	57	0.991

FP: False Positives; FN: False Negatives.

while dynamically enhancing its detection performance through external knowledge integration.

5. Conclusion

To address critical bottlenecks associated with the semantic modeling and interpretability of the existing methods, the present study proposed LLM-LADE, a framework that accomplishes log anomaly detection and explanatory generation through a three-stage collaborative architecture. First, the few-shot learning capability of a GPT-4 model was leveraged to generate label-cause pairs. Subsequently, the LLaMA3-8B model was adapted using the LoRA technique. To tackle the class imbalance issue in log data, a hybrid loss function was proposed, incorporating both a focal loss to enhance focus on anomalous samples and a dynamic weighting coefficient α to alleviate the interference of long-text generation on label learning. Finally, an incremental knowledge base was constructed to capture misjudged samples in real time, achieving dynamic performance optimization with zero parameter updates via a matching-triggered priority retrieval mechanism in the knowledge base.

Evaluations on three public real-world datasets demonstrated the superiority of LLM-LADE over baseline models. Ablation studies confirmed the effectiveness of the hybrid loss function and the online performance enhancement enabled by the feedback mechanism. Both the GPT-4 model and human expert evaluations validated that explanations generated achieved professional-grade usability in terms of usefulness and readability. Finally, quantitative analysis revealed the marginal contribution of log parsing to detection performance, proving the inherent capability of LLMs to autonomously extract semantic features from unstructured logs.

Future work is expected to focus on the optimization of the semantic retrieval mechanisms of the knowledge base and the development of lightweight deployment schemes to further enhance the engineering applicability of the framework. Furthermore, comprehensive validation will be conducted on large-scale software systems and extended to diverse log datasets.

CRediT authorship contribution statement

Zhiwei Zhang: Methodology, Writing – original draft; **Saifei Li:** Conceptualization, Writing – review & editing; **Lijie Zhang:** Resources; **Jianbin Ye:** Validation; **Chunduo Hu:** Investigation; **Lianshan Yan:** Supervision.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Saifei Li reports financial support was provided by The National Natural Science Foundation of China under Grant No. 62301461. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grant no. 62301461.

References

[1] J. Ko, M. Comuzzi, A systematic review of anomaly detection for business process event logs, Bus. Inf. Syst. Eng. 65 (2023) 441–462. https://doi.org/10.1007/s12599-023-00794-y
[2] V.-H. Le, H. Zhang, Log-based anomaly detection with deep learning: how far are we? in: Proceedings of the 44th International Conference on Software Engineering (ICSE '22), Pittsburgh, PA, USA, ACM, 2022, pp. 1356–1367. https://doi.org/10.1145/3510003.3510155
[3] Z. Chen, J. Liu, W. Gu, Y. Su, M.R. Lyu, Experience Report: Deep Learning-based System Log Analysis for Anomaly Detection, arXiv preprint arXiv:2107.05908, 2021.
[4] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9(8) (1997) 1735–1780.
[5] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: pre-training of deep bidirectional transformers for language understanding, in: Proc. 2019 Conf. North Am Chapter Assoc. Comput. Linguist.: Hum. Lang. Technol. (NAACL-HLT '19), 2019, pp. 4171–4186. .
[6] Y. Fu, K. Liang, J. Xu, Mlog: mogrifier lstm-based log anomaly detection approach using semantic representation, IEEE Trans. Serv. Comput. 16(5) (2023) 3537–3549.
[7] F. Hadadi, Q. Xu, D. Bianculli, L. Briand, Anomaly detection on unstable logs with gpt models, arXiv preprint arXiv:2406.07467, 2024.
[8] X. Zhang, Y. Xu, Q. Lin, B. Qiao, H. Zhang, Y. Dang, C. Xie, X. Yang, Q. Cheng, Z. Li, Others, Robust log-based anomaly detection on unstable log data, in: Proc. 27th ACM Joint Meet. Eur. Softw. Eng. Conf. & Symp. Found. Softw. Eng. (ESEC/FSE '19), 2019, pp. 807–817.
[9] L. Yang, J. Chen, Z. Wang, W. Wang, J. Jiang, X. Dong, W. Zhang, Semi-supervised log-based anomaly detection via probabilistic label estimation, in: Proc. 43rd IEEE/ACM Int. Conf. Softw. Eng. (ICSE '21), 2021, pp. 1448–1460.
[10] S. Nedelkoski, J. Bogatinovski, A. Acker, J. Cardoso, O. Kao, Self-attentive classification-based anomaly detection in unstructured logs, in: Proc. 2020 IEEE Int. Conf. Data Min. (ICDM '20), 2020, pp. 1196–1201.
[11] H. Guo, J. Yang, J. Liu, J. Bai, B. Wang, Z. Li, T. Zheng, B. Zhang, J. Peng, Q. Tian, Logformer: a pre-train and tuning pipeline for log anomaly detection, Proc. AAAI Conf. Artif. Intell. (AAAI '24) vol. 38 (2024) 135–143.
[12] C. Zhang, X. Wang, H. Zhang, J. Zhang, H. Zhang, C. Liu, P. Han, Layerlog: log sequence anomaly detection based on hierarchical semantics, Appl. Soft Comput. 132 (2023) 109860.
[13] Z. Yin, X. Kong, C. Yin, Semi-supervised log anomaly detection based on bidirectional temporal convolution network, Comput. Secur. 140 (2024) 103808.
[14] M. Du, F. Li, G. Zheng, V. Srikumar, Deeplog: anomaly detection and diagnosis from system logs through deep learning, in: Proc. 2017 ACM SIGSAC Conf. Comput. Commun. Secur. (CCS '17), 2017, 2017, pp. 1285–1298.
[15] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun, Loganomaly, Unsupervised detection of sequential and quantitative anomalies in unstructured logs, Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI '19) vol. 19 (2019) 4739–4745.
[16] H. Guo, S. Yuan, X. Wu, Logbert: log anomaly detection via bert, in: Proc. 2021 Int. Joint Conf. Neural Netw. (IJCNN '21), 2021, pp. 1–8.
[17] Y. Lee, J. Kim, P. Kang, Lanobert: system log anomaly detection based on bert masked language model, Appl. Soft Comput. 146 (2023) 110689.
[18] C. Almodovar, F. Sabrina, S. Karimi, S. Azad, Logfit: log anomaly detection using fine-tuned language models, IEEE Trans. Netw. Serv. Manag. 21(2) (2024) 1715–1723.
[19] Y. Lin, H. Deng, X. Li, Fastlogad, Log Anomaly Detection with Mask-Guided Pseudo Anomaly Generation and Discrimination, arXiv preprint arXiv:2404.08750 2024.
[20] V.-H. Le, H. Zhang, Log-based anomaly detection without log parsing, in: Proc. 36th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE '21), 2021, pp. 492–504.
[21] Z. Li, Y. Zhu, M.V. Leeuwen, A survey on explainable anomaly detection, ACM Trans. Knowl. Discov. Data 18(1) (2023) 1–54.
[22] J. Wang, G. Chu, J. Wang, H. Sun, Q. Qi, Y. Wang, J. Qi, J. Liao, Logexpert: log-based recommended resolutions generation using large language model, in: Proc. 44th ACM/IEEE Int. Conf. Softw. Eng.: New Ideas Emerg. Results (ICSE '24 NIER), 2024, pp. 42–46.
[23] Z. Li, J. Shi, M.V. Leeuwen, Graph neural networks based log anomaly detection and explanation, in: Proc. 46th IEEE/ACM Int. Conf. Softw. Eng. Companion (ICSE '24 Companion), 2024, pp. 306–307.
[24] Y. Nie, Y. Kong, X. Dong, J.M. Mulvey, H.V. Poor, Q. Wen, S. Zohren, A survey of large language models for financial applications: progress, prospects and challenges, arXiv preprint arXiv:2406.11903, 2024.
[25] N. Neveditsin, P. Lingras, V. Mago, Clinical insights: a comprehensive review of language models in medicine, arXiv preprint arXiv:2408.11735 2024.
[26] W. Gan, Z. Qi, J. Wu, J.C.W. Lin, Large language models in education: vision and opportunities, in: 2023 IEEE International Conference on Big Data (BigData), IEEE, 2023, pp. 4776–4785.
[27] J. Qi, S. Huang, Z. Luan, S. Yang, C. Fung, H. Yang, D. Qian, J. Shang, Z. Xiao, Z. Wu, Loggpt, Exploring chatgpt for log-based anomaly detection, in: 2023 IEEE Int. Conf. High Performance Comput, IEEE, 2023, pp. 273–280.

[28] Y. Liu, S. Tao, W. Meng, F. Yao, X. Zhao, H. Yang, Logprompt: prompt engineering towards zero-shot and interpretable log analysis, in: Proc. 46th IEEE/ACM Int. Conf. Softw. Eng. Companion (ICSE '24 Companion), 2024, pp. 364–365.
[29] C. Egersdoerfer, D. Zhang, D. Dai, Early exploration of using chatgpt for log-based anomaly detection on parallel file systems logs, in: Proc. 32nd Int. Symp. High-Perform. Parallel Distrib. Comput. (HPDC '23), 2023, pp. 315–316.
[30] H. Guo, J. Yang, J. Liu, L. Yang, L. Chai, J. Bai, J. Peng, X. Hu, C. Chen, D. Zhang, Owl, A large language model for it operations, arXiv preprint arXiv:2309.09298, 2023.
[31] Y.F. Lim, J. Zhu, G. Pang, Adapting Large Language Models for Parameter-Efficient Log Anomaly Detection, 2503.08045, 2025.
[32] S. Song, Y. Zhang, N. Gao, Confront insider threat: precise anomaly detection in behavior logs based on LLM fine-tuning, in: Proceedings of the 31st International Conference on Computational Linguistics, 2025, pp. 8589–8601.
[33] W. Guan, J. Cao, S. Qian, J. Gao, C. Ouyang, LogLLM: Log-based Anomaly Detection Using Large Language Models, arXiv preprint arXiv:2411.08561, 2024.
[34] G. No, Y. Lee, H. Kang, P. Kang, Training-free retrieval-based log anomaly detection with pre-trained language model considering token-level information, Eng. Appl. Artif. Intell. 133 (2024) 108613.
[35] Z. Jiang, J. Liu, J. Huang, Y. Li, Y. Huo, J. Gu, Z. Chen, J. Zhu, M.R. Lyu, A large-scale evaluation for log parsing techniques: how far are we?, Proc. 33rd ACM SIGSOFT Int. Symp. Softw. Test. Anal. (ISSTA '24), (2024) pp. 223–234.
[36] P. He, J. Zhu, Z. Zheng, M.R. Lyu, Drain: an online log parsing approach with fixed depth tree, in: Proc. 2017 IEEE Int. Conf. Web Serv. (ICWS '17), 2017, pp. 33–40.
[37] S. Yu, P. He, N. Chen, Y. Wu, Brain: log parsing with bidirectional parallel tree, IEEE Trans. Serv. Comput. 16(5) (2023) 3224–3237.
[38] V.-H. Le, H. Zhang, Log parsing with prompt-based few-shot learning, in: Proc. 45th IEEE/ACM Int. Conf. Softw. Eng. (ICSE '23), 2023, pp. 2438–2449.
[39] Y. Liu, X. Zhang, S. He, H. Zhang, L. Li, Y. Kang, Y. Xu, M. Ma, Q. Lin, Y. Dang, Others, Uniparser: a unified log parser for heterogeneous log data, in: Proc. ACM Web Conf. (WWW '22), 2022, pp. 1893–1901.
[40] I. Sedki, A. Hamou-Lhadj, O. Ait-Mohamed, M.A. Shehab, An effective approach for parsing large log files, in: Proc. 2022 IEEE Int. Conf. Softw. Maint. Evol. (ICSME '22), 2022, pp. 1–12.
[41] W. Xu, L. Huang, A. Fox, D. Patterson, M.I. Jordan, Detecting large-scale system problems by mining console logs, in: Proc. ACM SIGOPS 22nd Symp. Oper. Syst. Princ. (SOSP '09), 2009, pp. 117–132.
[42] J.-G. Lou, Q. Fu, S. Yang, Y. Xu, J. Li, Mining invariants from console logs for system problem detection, in: Proc. 2010 USENIX Annu. Tech. Conf. (USENIX ATC '10), 2010.
[43] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, Roberta: a robustly optimized bert pretraining approach, 1907.11692, 2019.
[44] A. Joulin, E. Grave, P. Bojanowski, M. Douze, H. Jégou, T. Mikolov, FastText.zip: Compressing Text Classification Models, arXiv preprint arXiv:1612.03651, 2016.
[45] Y. Li, Y. Liu, H. Wang, Z. Chen, W. Cheng, Y. Chen, W. Yu, H. Chen, C. Liu, Glad: content-aware dynamic graphs for log anomaly detection, in: Proc. 2023 IEEE Int. Conf. on Knowledge Graph (ICKG), 2023, pp. 9–18.
[46] S. Tan, D. Li, R. Jiang, Y. Zhang, M. Okumura, Community-invariant graph contrastive learning, in: Proceedings of the 41st International Conference on Machine Learning (ICML), 2024, pp. 47579–47606.
[47] D. Li, S. Tan, Y. Wang, K. Funakoshi, M. Okumura, Temporal and topological augmentation-based cross-view contrastive learning model for temporal link prediction, in: Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM), 2023, pp. 4059–4063.
[48] J. Pan, W.S. Liang, Y. Yidi, Raglog: log anomaly detection using retrieval augmented generation, in: Proc. 2024 IEEE World Forum Public Saf. Technol. (WFPST '24), 2024, pp. 169–174.
[49] N. Ding, Y. Qin, G. Yang, F. Wei, Z. Yang, Y. Su, S. Hu, Y. Chen, C.-M. Chan, W. Chen, Parameter-efficient fine-tuning of large-scale pre-trained language models, Nat. Mach. Intell. 5(3) (2023) 220–235.
[50] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, Focal loss for dense object detection, in: Proc. IEEE Int. Conf. Comput. Vis. (ICCV '17), 2017, pp. 2980–2988.
[51] V. Mittal, I. Kashyap, An overview of real world applications with concept drifting data streams, in: Proc. 3rd Int. Conf. Internet Things Connected Technol. (ICIoTCT '18), 2018, pp. 26–27.
[52] J. Zhu, S. He, P. He, J. Liu, M.R. Lyu, Loghub: a large collection of system log datasets for ai-driven log analytics, in: Proc. 34th IEEE Int. Symp. Softw. Rel. Eng. (ISSRE '23), 2023, pp. 355–366.
[53] W. Xu, L. Huang, A. Fox, D. Patterson, M. Jordan, Online system problem detection by mining patterns of console logs, in: Proc. 9th IEEE Int. Conf. Data Min. (ICDM '09), 2009, pp. 588–597.
[54] A. Oliner, J. Stearley, What supercomputers say: a study of five system logs, in: Proc. 37th IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN '07), 2007, pp. 575–584.
[55] I. Loshchilov, F. Hutter, Decoupled Weight Decay Regularization, arXiv preprint arXiv:1711.05101, 2017.
[56] Y. Chen, R. Wang, H. Jiang, S. Shi, R. Xu, Exploring the use of large language models for reference-free text quality evaluation: an empirical study, arXiv preprint arXiv:2304.00723, 2023.
[57] H. Song, H. Su, I. Shalyminov, J. Cai, S. Mansour, Finesure: fine-grained summarization evaluation using llms, In: Proc. 62nd Annu. Meeting of the Association for Computational Linguistics (ACL), Bangkok, Thailand, 2024, pp. 906–922. https://doi.org/10.18653/v1/2024.acl-long.51
[58] S.Y. Chu, J.W. Kim, M.Y. Yi, Think together and work better: combining humans' and LLMs' think-aloud outcomes for effective text evaluation, in: Proc. 2025 CHI Conf. on Human Factors in Computing Systems (CHI), 2025, pp. 1–23.
[59] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, S. Rajmohan, Recommending root-cause and mitigation steps for cloud incidents using large language models, in: Proc. 45th IEEE/ACM Int. Conf. Softw. Eng. (ICSE '23), 2023, pp. 1737–1749.